



Recibe una cálida:

# ¡Bienvenida!

---

Te estábamos esperando 😊 +

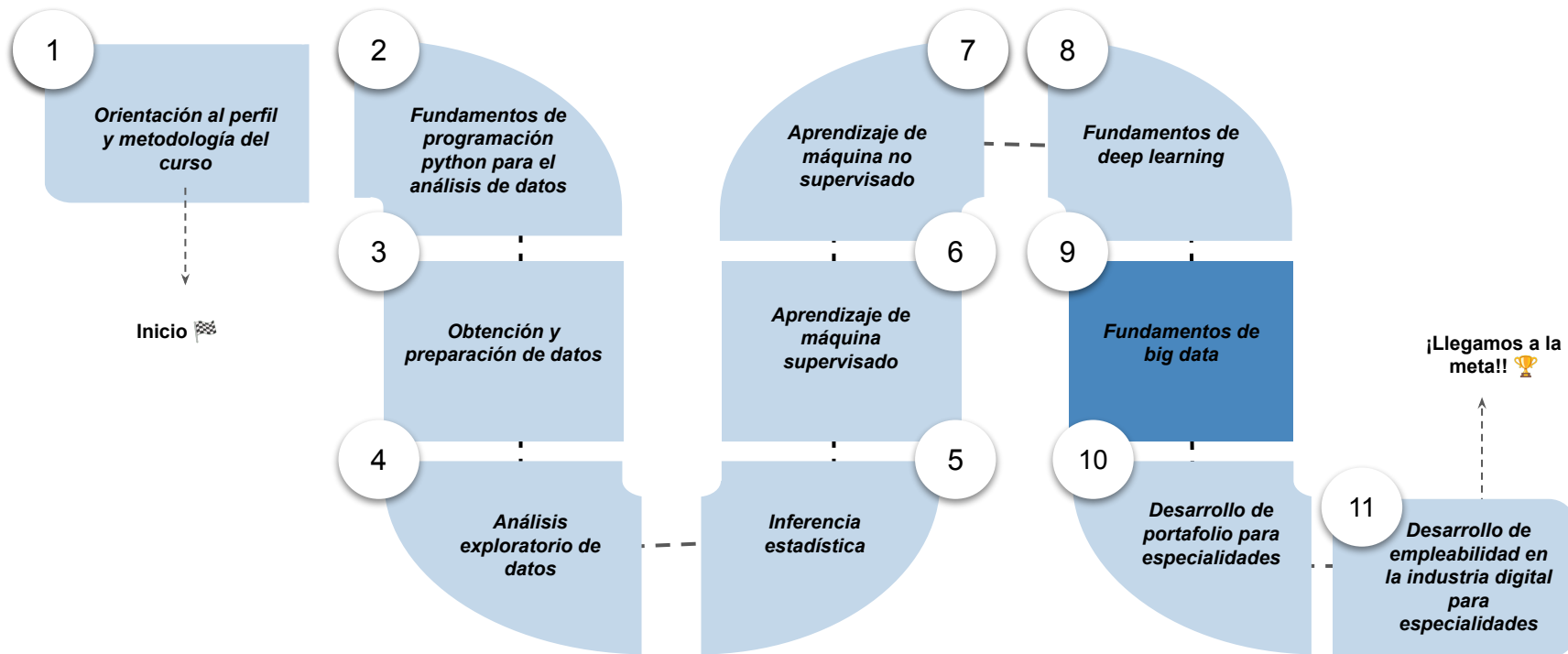


# › Elementos básicos de spark

**Aprendizaje Esperado 3:** Manipular grandes volúmenes de datos utilizando objetos rdd, transformaciones y acciones para resolver un problema en el entorno spark.

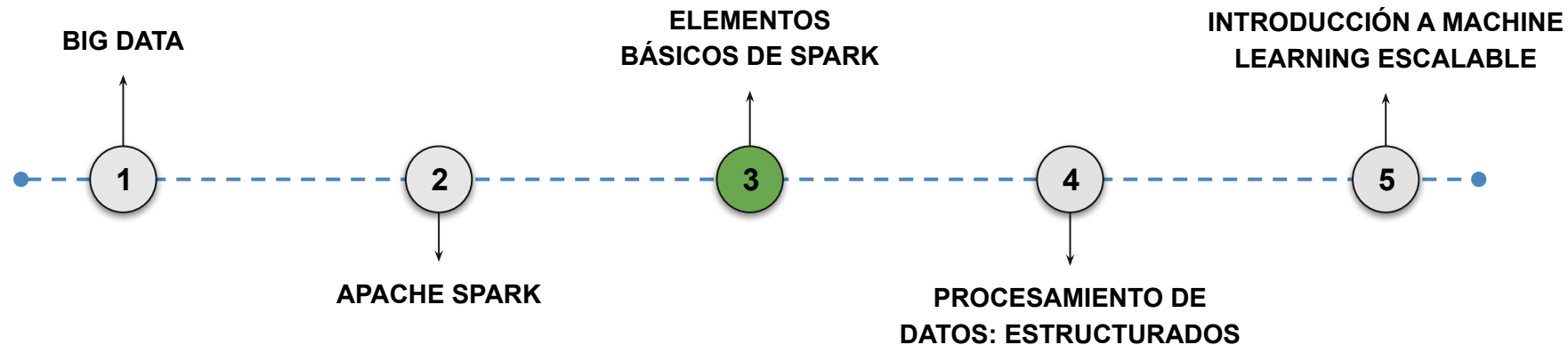
# Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



# Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



# Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Manipulación de datos con  
RDD, transformaciones y  
acciones en Spark

*Aprenderemos a utilizar los elementos básicos de Spark para trabajar con grandes volúmenes de datos, desde la creación de RDD hasta la aplicación de transformaciones y acciones para obtener resultados.*

Fundamentos de RDD y  
su uso en Spark

Qué es un RDD y cómo se crea.

Persistencia y cacheo de datos.

Transformaciones y  
acciones en RDD

Tipos y ejemplos de transformaciones: map, filter, flatMap, reduceByKey, join.

Acciones: collect, take, count, sum, mean.

# Learning Path

¿Cuáles temas trabajaremos hoy?

3.

## Optimización y funcionalidades avanzadas con RDDs en Spark

Profundizaremos en técnicas para optimizar el uso de RDDs, trabajar con datos en distintos formatos y aprovechar funcionalidades avanzadas de Spark como variables broadcast, acumuladores y particionamiento personalizado.

Optimización y buenas prácticas en RDDs

Diferencia entre groupByKey y reduceByKey.

Minimizar shuffle y mejorar rendimiento.

Funcionalidades y extensiones de Spark

Variables broadcast y acumuladores.

Lectura y escritura en múltiples formatos (CSV, JSON, Parquet).

# Objetivos de aprendizaje

*¿Qué aprenderás?*

- Comprender qué es un RDD y su rol en Spark.
- Crear, persistir y manipular RDDs desde distintas fuentes.
- Aplicar transformaciones para preparar y filtrar datos.
- Ejecutar acciones para obtener resultados concretos.
- Entender el concepto de linaje y lazy evaluation para optimizar procesos.



# Objetivos de aprendizaje

*¿Qué aprenderás?*



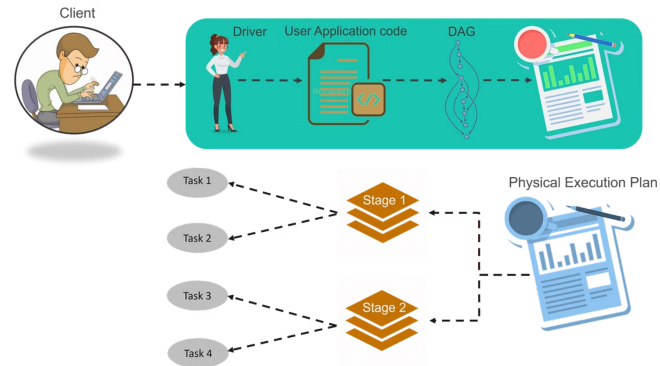
- Optimizar operaciones en RDD minimizando shuffle.
- Utilizar variables broadcast y acumuladores para mejorar el rendimiento.
- Leer y escribir datos en distintos formatos desde Spark.
- Integrar RDD con Spark SQL y otros módulos.
- Aplicar buenas prácticas de monitoreo, depuración y rendimiento.



# Elementos básicos de spark

# Apache Spark: Fundamentos y Procesamiento de Datos Distribuido

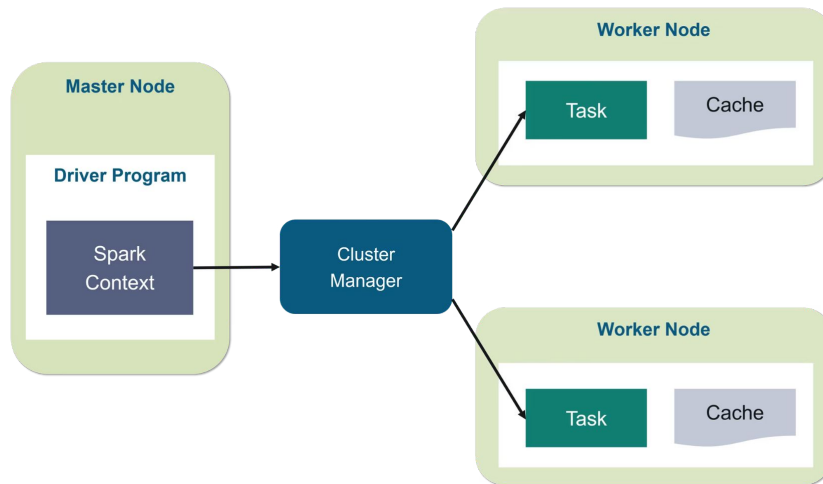
Apache Spark es uno de los motores de procesamiento de datos más potentes y utilizados en la actualidad para manejar grandes volúmenes de información. Su principal característica es la capacidad de realizar análisis masivo de datos de forma distribuida y en memoria, lo que permite obtener resultados de manera más rápida que sistemas tradicionales como Hadoop MapReduce.

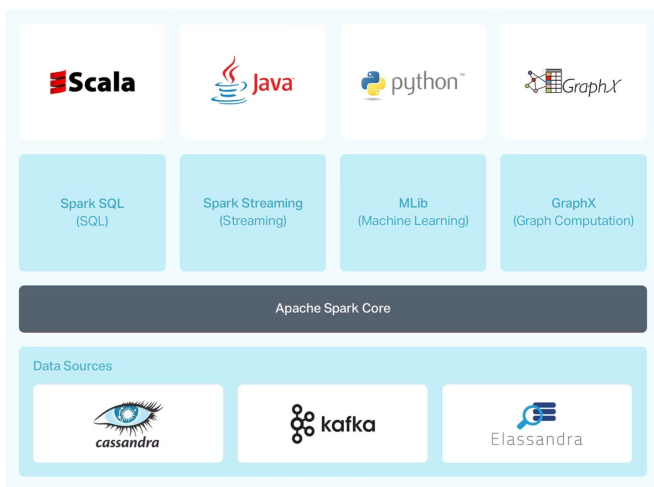


# Introducción a Apache Spark

Para trabajar con Spark es fundamental conocer sus componentes básicos, entre los que se destacan los **RDD** (Resilient Distributed Datasets), las **transformaciones** y las **acciones**, que permiten manipular datos a gran escala.

También es esencial entender cómo funciona la configuración inicial, el contexto de Spark y los conceptos de **linaje** y **evaluación perezosa (lazy evaluation)**, los cuales impactan directamente en la eficiencia del procesamiento.





# Elementos Básicos de Spark

Para dominar Apache Spark, es necesario comprender sus elementos fundamentales que permiten el procesamiento distribuido de datos a gran escala.

# Configuración, Conexión y Contexto de Spark

Para trabajar con Spark se debe inicializar un **SparkContext**, que es el punto de entrada para cualquier operación. A través del contexto, Spark administra recursos y coordina la ejecución distribuida.

```
from pyspark import SparkContext  
  
sc = SparkContext("local", "MiAplicacionSpark")  
  
print(sc)
```

# Componentes de la Configuración

## Configuración local

local indica que el procesamiento se ejecutará en el equipo local.

## Nombre de la aplicación

Útil para identificar el job en el monitor de Spark.

## Driver y ejecutores

Spark coordina tareas mediante un driver y múltiples ejecutores distribuidos.

Tener correctamente configurado el contexto garantiza que las operaciones sobre datos se ejecuten de manera eficiente en clústeres grandes o entornos de prueba locales.

# RDD: Qué es un RDD

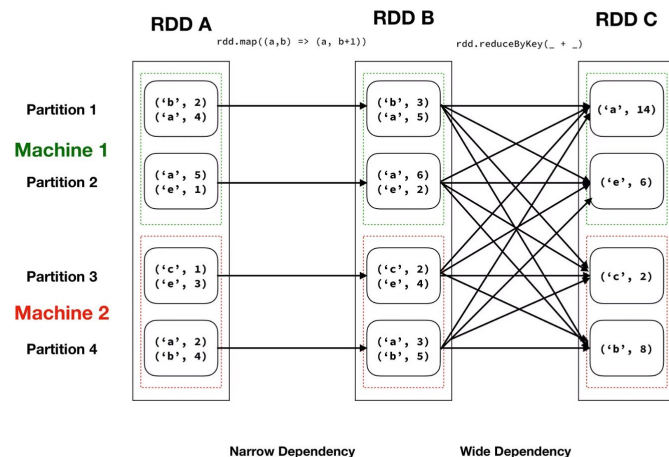
Un **RDD (Resilient Distributed Dataset)** es la estructura de datos fundamental de Spark.

Se trata de un conjunto de elementos distribuidos a través de varios nodos de un clúster, que permite:

**Procesamiento paralelo:** cada partición se procesa de manera independiente.

**Tolerancia a fallos:** puede reconstruirse a partir de su linaje si un nodo falla.

**Evaluación diferida:** las operaciones no se ejecutan inmediatamente, sino cuando se requiere un resultado final.



# Creación y Carga de un RDD

Podemos crear un RDD de distintas maneras:

## 1. Desde una colección en Python

---

```
datos = [1, 2, 3, 4, 5]  
rdd = sc.parallelize(datos)
```

---

## 2. Desde una base de datos o fuente externa

---

```
rdd = sc.textfile("ruta/file.txt")
```

---



# Almacenamiento de un RDD

---

```
rdd_filtrado = rdd.filter(lambda x: x > 10)
```

```
rdd_filtrado.count()  
rdd_filtrado.collect()
```

---

Spark ejecuta el filtro dos veces

Solución:

## **persist()**

Permite elegir cómo almacenar (memoria, disco, etc.)

```
rdd_filtrado.persist()
```

### **Persistencia en memoria**

`rdd.persist()` mantiene los datos para múltiples usos.

## **cache()**

Guarda el RDD en memoria (por defecto)

```
rdd_filtrado.cache()
```

### **Caché**

`rdd.cache()` es equivalente a persistencia en memoria por defecto.



Q Comandos + Código ▾ + Texto ▶ Ejecutar todas ▾



## • Sin cache

```
[3]
✓ 3 s

from pyspark import SparkContext
# sc = SparkContext("local", "Ejemplo")

datos = [5, 15, 25, 8, 30]

rdd = sc.parallelize(datos)

rdd_filtrado = rdd.filter(lambda x: x > 10)

print(rdd_filtrado.count()) # acción 1
print(rdd_filtrado.collect()) # acción 2
```

✓ 3  
[15, 25, 30]

## • Con cache

```
[4]
✓ 1 s

▶ rdd_filtrado = rdd.filter(lambda x: x > 10)

rdd_filtrado.cache()

print(rdd_filtrado.count()) # ejecuta y guarda
print(rdd_filtrado.collect()) # usa lo guardado
```

✓ ... 3  
[15, 25, 30]

# Pair-RDD

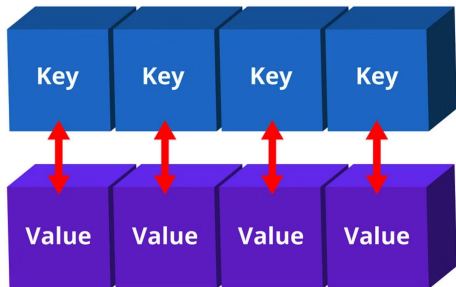
Un Pair RDD es un RDD donde cada elemento tiene la forma: (**clave**, **valor**)

```
rdd = sc.parallelize([("a", 1), ("b", 2), ("a", 3)])
```

## ¿Para qué sirve?

- \* Agrupar datos por clave
- \* Realizar agregaciones

Key-Value Pairs



```
[5] ✓ 0s rdd = sc.parallelize([("a", 1), ("b", 2), ("a", 3)])
```

---

```
[6] ✓ 0s ▶ rdd.reduceByKey(lambda x, y: x + y).collect()
```

```
✓ ... [('a', 4), ('b', 2)]
```

Este tipo de RDD es fundamental para operaciones de agregación (`reduceByKey`, `groupByKey`) y es ampliamente usado en análisis de grandes volúmenes de datos.

# Linaje y Lazy Evaluation

## Linaje

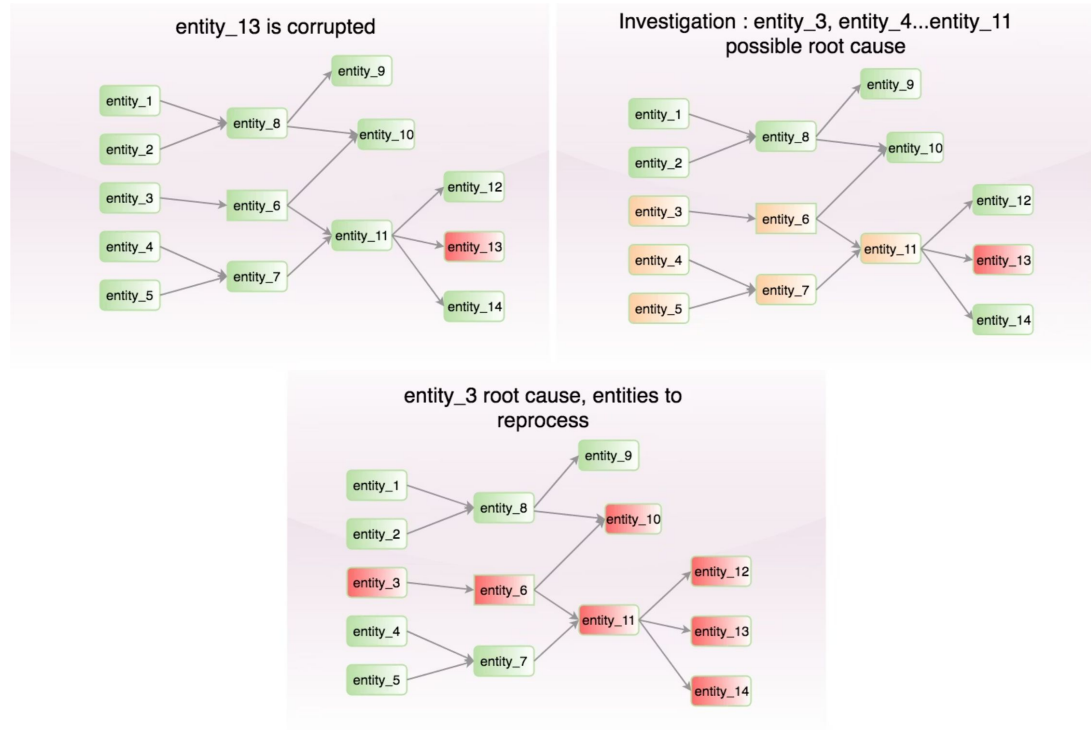
Registro de las transformaciones aplicadas a un RDD.

## Lazy Evaluation

Spark no ejecuta las transformaciones inmediatamente; espera hasta que se invoque una **acción**.

Este enfoque permite optimizaciones automáticas y evita cálculos innecesarios.

*Lineage = historial de cómo se construyen los datos*  
*Lazy = Spark no ejecuta hasta que se lo pides*



# Transformaciones

Una **transformación** es una operación que crea un nuevo RDD a partir de uno existente.

Son operaciones **perezosas** (lazy), es decir, no ejecutan el cálculo hasta que se invoque una acción.

## Principales Métodos de Transformación



### **filter(func)**

Devuelve elementos que cumplen una condición.



### **map(func)**

Aplica una función a cada elemento del RDD.



### **flatMap(func)**

Igual que map, pero devuelve iteradores planos.



### **sample()**

Toma una muestra aleatoria del RDD.

# Más Métodos de Transformación



## **union()**

Une dos RDD en uno solo.



## **distinct()**

Elimina duplicados.



## **sortBy()**

Ordena el RDD según un criterio.

# Ejemplo de Transformaciones

En este ejemplo, se crea un RDD a partir de una lista de números, se aplica una transformación map para duplicar cada valor, y luego se aplica filter para quedarse solo con los valores mayores a 10.

---

```
▶ rdd = sc.parallelize([1, 2, 3, 4, 5])

    resultado = rdd.filter(lambda x: x % 2 == 0) \
                    .map(lambda x: x**2)

    print(resultado.collect()) # [4, 16]
```

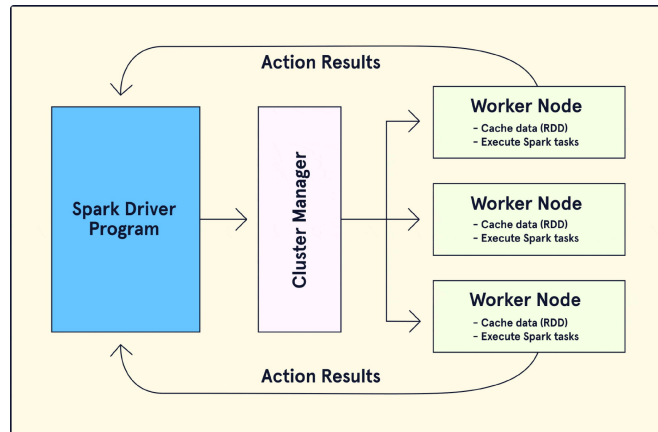
---

... [4, 16]

---

# Acciones

Una **acción** ejecuta realmente las transformaciones pendientes sobre un RDD y devuelve un resultado.





# Principales Acciones



## **collect()**

Devuelve todos los elementos como una lista.



## **take(n)**

Obtiene los primeros n elementos.



## **top(n)**

Obtiene los n elementos más altos según el orden natural.



## **takeSample()**

Extrae una muestra aleatoria.

# Más Acciones



## **sum()**

Suma todos los elementos  
(numéricos).



## **mean()**

Calcula la media de los elementos  
(requiere RDD numérico).



## **stdev()**

Calcula la desviación estándar de  
los datos.

# Ejemplo de Acciones

En este ejemplo, se crea un RDD con una lista de números, se aplica una transformación filter para seleccionar solo los números pares, y finalmente se ejecuta la acción collect() para obtener los resultados.

[8]  
✓ 0 s

```
▶ rdd = sc.parallelize([1, 2, 3, 4, 5])  
  
print(rdd.sum())      # 15  
print(rdd.take(3))    # [1, 2, 3]
```

▼

```
... 15  
    [1, 2, 3]
```

# ¿Qué es un Job Spark?

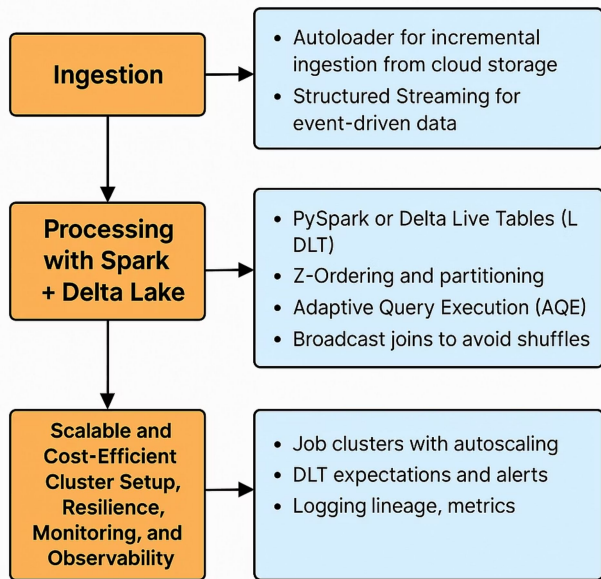
Un Job es el conjunto de tareas que Spark ejecuta cuando se invoca una **acción**

```
rdd.filter(...).map(...).collect()  
# => collect() dispara un Job
```

Las transformaciones no hacen nada...el Job empieza cuando pides el resultado

## Flujo

- 1 Se detecta una acción
- 2 Se construye el **DAG**
- 3 Se divide en **Stages**
- 4 Se divide en **Tasks**
- 5 Se ejecuta en paralelo
- 6 Se devuelven resultados



# Proceso de Ejecución de un Job

Cuando llamamos a `collect()`, `count()` u otra acción, Spark:



**Evalúa el linaje del RDD**



**Crea un plan de ejecución (DAG)**



**Divide el trabajo en stages y tasks**



**Retorna el resultado final al driver**

# Visualización de un Job Spark

Esta imagen muestra cómo se visualiza un Job Spark en la interfaz de usuario de Spark, con sus diferentes etapas y tareas.

**Acción (collect)**



**Job**



**Stage 1**

→ **lectura + filter**



**Stage 2**

→ **groupBy (shuffle)**



**Stage 3**

→ **resultado final**

# Importancia de los RDDs en Spark

El manejo de **RDDs en Spark** constituye la base del procesamiento de datos distribuido.

Comprender cómo crear, almacenar y transformar RDDs, junto con la diferencia entre **transformaciones** y **acciones**, permite construir soluciones escalables y eficientes.

## ¿Por qué son importantes los RDDs?

### Lo que permiten

- Procesar grandes volúmenes de datos
- Distribuir el trabajo en múltiples nodos
- Aplicar transformaciones y acciones

### Lo realmente importante

Son la base del modelo de ejecución de Spark

**Idea clave:** RDD es la forma en que Spark distribuye los datos

# Beneficios de Lazy Evaluation y Linaje

## Optimización

El concepto de **lazy evaluation** y **linaje** ofrece optimizaciones automáticas al planificar la ejecución de manera eficiente.

## ¿Por qué Lazy Evaluation y Lineage son clave?

### Lazy Evaluation

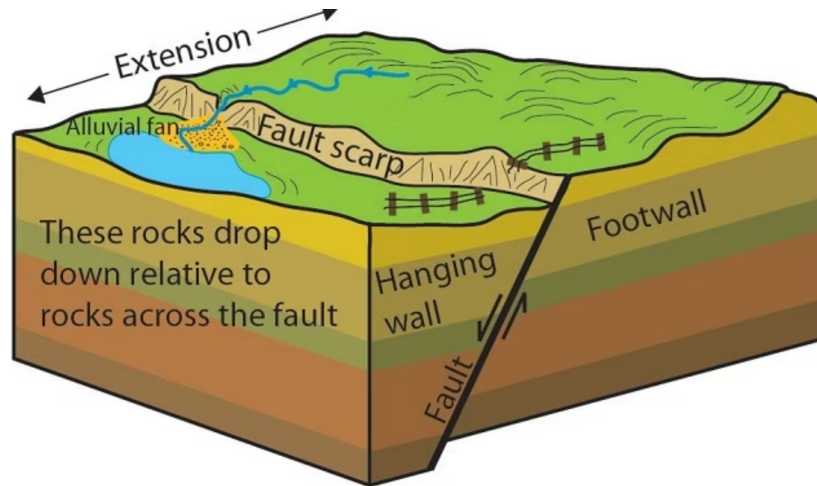
- No ejecuta hasta que hay acción
- Permite optimizar el plan

### Lineage

- Guarda el historial de transformaciones
- Permite recomputar datos si hay fallos

## Tolerancia a fallos

El linaje permite reconstruir datos perdidos en caso de fallos en los nodos del clúster.





# Entendiendo los Jobs de Spark

¿Qué es un Job en Spark?

## Definición clara

Un Job se crea cuando ejecutamos una **acción**

## Ejemplo

```
rdd.map(...).filter(...).collect()
```

`collect()` crea un Job

## Qué hace el Job

- Construye el DAG
- Lo divide en Stages
- Ejecuta Tasks en paralelo

**Idea clave:** El Job es cuando Spark pasa de plan a ejecución

# Profundizando en RDDs

## Particionamiento

Los RDDs se dividen en particiones que pueden procesarse en paralelo en diferentes nodos.

## Localidad

Spark intenta procesar datos cerca de donde están almacenados para reducir transferencias.



## Inmutabilidad

Los RDDs no pueden modificarse una vez creados, solo transformarse en nuevos RDDs.

## Computación

Cada RDD sabe cómo recalcul sus datos utilizando transformaciones anteriores.

RDD = datos distribuidos + procesamiento paralelo

## ¿Cómo guarda Spark los datos en memoria?

### Tipos de persistencia

| Tipo            | Qué hace               | Cuándo usar              |
|-----------------|------------------------|--------------------------|
| MEMORY_ONLY     | Guarda en RAM          | Datos pequeños / rápidos |
| MEMORY_AND_DISK | RAM + disco si no cabe | Caso más común           |
| DISK_ONLY       | Solo en disco          | Datos muy grandes        |
| MEMORY_ONLY_SER | En RAM comprimido      | Ahorra memoria           |

# Tipos de Persistencia en RDDs

## **MEMORY\_ONLY**

Almacena RDD como objetos Java deserializados en la JVM.

## **MEMORY\_AND\_DISK**

Almacena en memoria y, si no cabe, en disco.

## **MEMORY\_ONLY\_SER**

Almacena como bytes serializados (más eficiente en espacio).

## **DISK\_ONLY**

Almacena solo en disco, útil para conjuntos de datos muy grandes.

# Operaciones Avanzadas con Pair-RDDs



## **reduceByKey(func)**

Combina valores con la misma clave usando una función.

```
rdd.reduceByKey(lambda x, y: x + y)
```

*Suma valores por clave*



## **groupByKey()**

Agrupar valores con la misma clave.

```
rdd.groupByKey()
```

*Agrupar valores (menos eficiente)*



## **join(otherRDD)**

Une dos RDDs por clave, similar a un JOIN en SQL.

```
rdd1.join(rdd2)
```

*Une datasets por clave*



## **countByKey()**

Cuenta elementos por cada clave.

```
rdd.countByKey()
```

*Cuenta ocurrencias*

# Ejemplo de reduceByKey

```
[9] ✓ 0s ▶ # Crear un Pair-RDD
      pairs = sc.parallelize([("a", 1), ("b", 1), ("a", 1), ("a", 1), ("b", 1)])

      # Sumar valores por clave
      sums = pairs.reduceByKey(lambda a, b: a + b)

      # Resultado esperado: [("a", 3), ("b", 2)]
      print(sums.collect())

      ... [('a', 3), ('b', 2)]
```

Este ejemplo muestra cómo sumar valores asociados a la misma clave utilizando reduceByKey.

# Live Coding

## ¿En qué consistirá la Demo?

*Analizaremos paso a paso cómo manipular datos en Spark usando RDD, aplicando transformaciones y acciones para resolver un caso práctico.*

1. *Crear un RDD desde una colección y desde un archivo externo.*
2. *Aplicar transformaciones como `map()`, `filter()` y `reduceByKey()`.*
3. *Ejecutar acciones como `collect()` y `count()`.*
4. *Observar el linaje y cómo Spark optimiza con lazy evaluation.*
5. *Interpretar el ciclo de ejecución de un Job Spark (DAG → stages → tasks).*



Ejercicio

# Manipulando datos con RDD en Spark



# Manipulando datos con RDD en Spark

Contexto: 🙌

Vas a aplicar lo aprendido sobre RDD para manipular datos de gran tamaño de forma distribuida.

Consigna: 📝

1. Crea un RDD desde un archivo de texto creado desde 0.
2. Aplica al menos tres transformaciones y dos acciones sobre el RDD.
3. Observa el linaje y explica cómo Spark ejecutó el Job.
4. Identifica cómo el lazy evaluation impactó en el resultado y optimización.

Paso a paso: ⚙️

1. Inicializa el contexto de Spark.
2. Carga datos y crea un RDD.
3. Aplica transformaciones para filtrar y transformar los datos.
4. Ejecuta acciones para obtener métricas o resultados.
5. Documenta tu flujo y conclusiones.

Tiempo 🕒: 45 Minutos



Momento:

# Time-out!

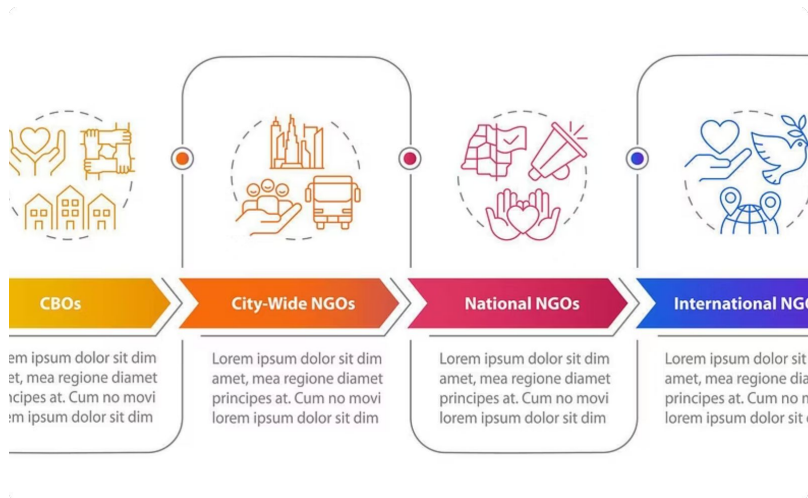


5 -10 min.



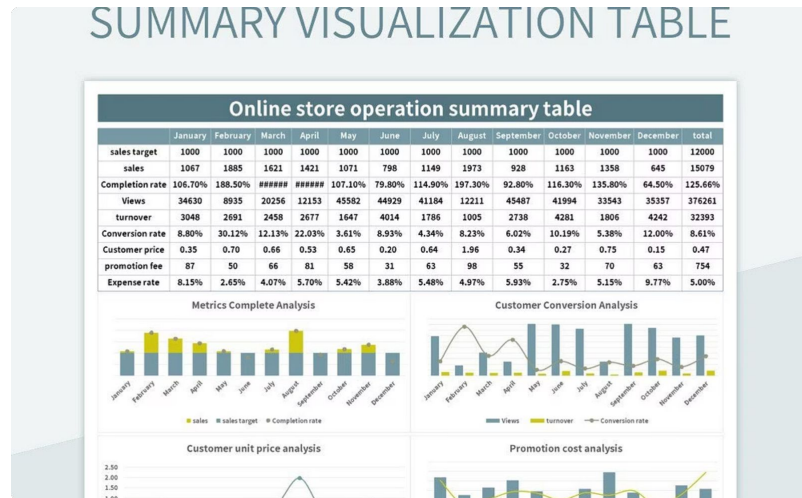
# Elementos básicos de spark

# Diferencia entre groupByKey y reduceByKey



## reduceByKey

Combina datos localmente antes de transferirlos, lo que reduce significativamente el tráfico de red.



## groupByKey

Transfiere todos los valores a través de la red y luego los agrupa, lo que puede causar problemas de rendimiento con grandes conjuntos de datos.

# Optimización de Shuffle en Spark

¿Qué es el shuffle?

Redistribución de datos entre particiones

Ocurre en operaciones como:

- groupByKey
- reduceByKey
- join

¿Por qué es costoso?

- Transferencia por red 
- Escritura/lectura en disco 
- Serialización de datos 

Puede afectar mucho el rendimiento

¿Cómo optimizarlo?

Usar reduceByKey en lugar de groupByKey

Reduce datos antes del shuffle

# Optimización de Shuffle en Spark

El **shuffle** es la redistribución de datos entre particiones que ocurre durante operaciones como groupByKey, reduceByKey o join.

## Impacto en el rendimiento

El shuffle es costoso porque implica E/S de disco, serialización y transferencia de red.

## Estrategias de optimización

Usar reduceByKey en lugar de groupByKey cuando sea posible.

## Particionamiento

Utilizar partitionBy para controlar la distribución de datos y minimizar shuffles.

# Broadcast Variables

Las **variables broadcast** permiten mantener una copia de solo lectura de una variable en cada nodo del clúster, en lugar de enviarla con cada tarea.

```
[10]  
✓ 0 s  
# Crear variable broadcastbroadcast  
Var = sc.broadcast([1, 2, 3])  
# Acceder al valorbroadcast  
Var.value  
  
[1, 2, 3]
```



Son útiles para compartir grandes conjuntos de datos de referencia que no cambian durante la ejecución.

# Acumuladores

Los **acumuladores** son variables que se pueden "acumular", es decir, agregar mediante una operación asociativa.

```
[ ] ▶ # Crear acumulador
      accum = sc.accumulator(0)

      # Función que incrementa el acumulador
      def f(x):
          global accum
          accum += x
          return x

      # Aplicar función a cada elemento
      rdd.foreach(f)

      # Valor final del acumulador
      print(accum.value)
```

Son útiles para contadores y sumas en operaciones distribuidas.



# Particionamiento Personalizado

Spark permite definir cómo se distribuyen los datos entre las particiones mediante el particionamiento personalizado.

[ ]



```
# Crear un RDD con particionamiento personalizado  
rdd = pairs.partitionBy(4, lambda key: hash(key) % 4)
```

Un buen particionamiento puede reducir significativamente el shuffle y mejorar el rendimiento.

# Lectura de Datos desde Archivos

T

## Archivos de texto

```
lines = sc.textFile("datos.txt")
```



## JSON

```
from pyspark.sql import SparkSession  
spark = SparkSession.builder.getOrCreate()  
df = spark.read.json("datos.json")
```



## CSV

```
df = spark.read.csv("datos.csv", header=True)
```

19

## Parquet

```
df = spark.read.parquet("datos.parquet")
```

# Escritura de Datos a Archivos



## Texto

```
rdd.saveAsTextFile("salida")
```



## SequenceFile

```
rdd.saveAsSequenceFile("salida")
```



## Hadoop

```
rdd.saveAsHadoopFile("salida")
```



## DataFrames

```
df.write.parquet("salida.parquet")
```

# Spark SQL: Integración con RDDs

Spark SQL permite trabajar con datos estructurados utilizando SQL dentro de aplicaciones Spark.

## Spark SQL: Integración con RDDs

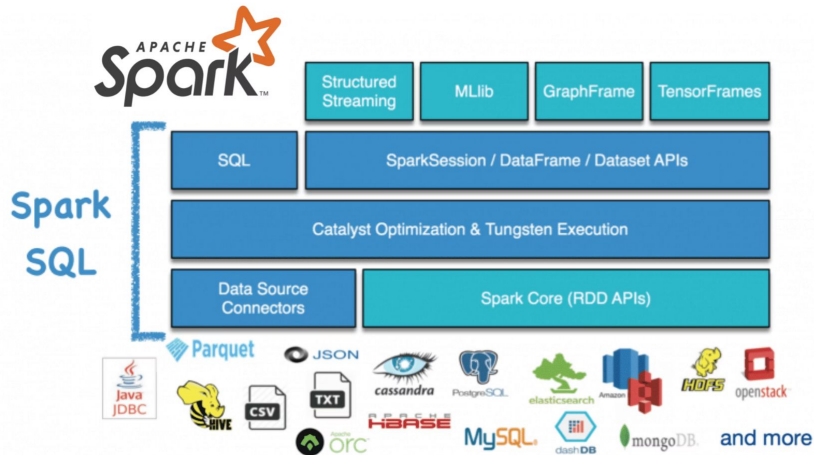
```
# Crear DataFrame desde RDD
from pyspark.sql import Row

Person = Row("name", "age")
people = rdd.map(lambda r: Person(r[0], r[1]))

df = spark.createDataFrame(people)

# Registrar como tabla temporal
df.createOrReplaceTempView("people")

# Ejecutar consulta SQL
result = spark.sql("SELECT * FROM people WHERE age > 20")
```



# Spark Streaming: Procesamiento en Tiempo Real

Spark Streaming permite procesar datos en tiempo real desde diversas fuentes como Kafka, Flume o TCP sockets.

## Spark Streaming: Procesamiento en Tiempo Real

```
[ ]  
from pyspark import SparkContext  
from pyspark.streaming import StreamingContext  
  
# Crear SparkContext  
sc = SparkContext("local[2]", "StreamingDemo")  
  
# Crear contexto de streaming con intervalo de 1 segundo  
ssc = StreamingContext(sc, 1)  
  
# Crear DStream desde socket  
lines = ssc.socketTextStream("localhost", 9999)  
  
# Contar palabras  
words = lines.flatMap(lambda line: line.split(" "))  
pairs = words.map(lambda word: (word, 1))  
wordCounts = pairs.reduceByKey(lambda a, b: a + b)  
  
# Imprimir resultados  
wordCounts.pprint()  
  
# Iniciar procesamiento  
ssc.start()  
ssc.awaitTermination()
```

# MLlib: Biblioteca de Machine Learning

## Algoritmos disponibles

- Clasificación: SVM, regresión logística, Naive Bayes
- Regresión: regresión lineal, árboles de decisión
- Clustering: K-means, LDA
- Recomendación: filtrado colaborativo

## Ejemplo básico

```
from pyspark.mllib.classification import LogisticRegressionWithSGD
from pyspark.mllib.regression import LabeledPoint

# Crear datos de entrenamiento
data = [
    LabeledPoint(0.0, [0.0, 1.0]),
    LabeledPoint(1.0, [1.0, 0.0])
]

rdd = sc.parallelize(data)

# Entrenar modelo
model = LogisticRegressionWithSGD.train(rdd)

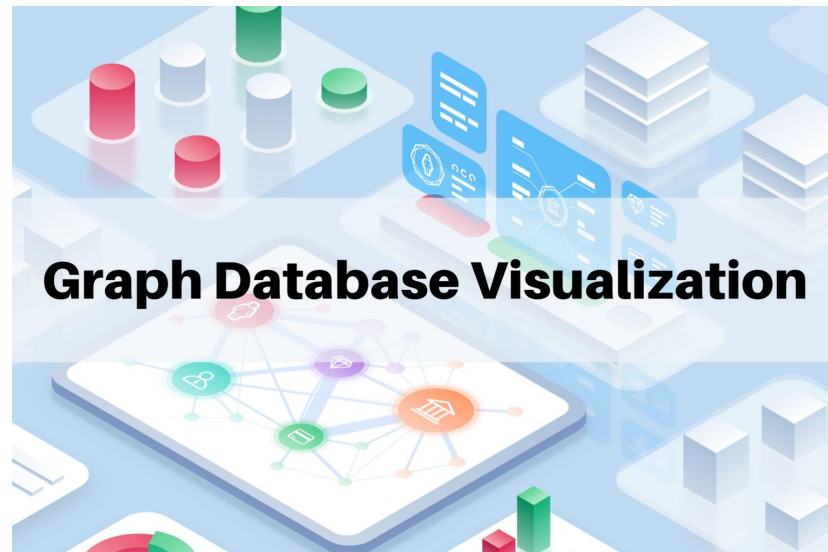
# Predecir
print(model.predict([1.0, 0.0]))
```

# GraphX: Procesamiento de Grafos

GraphX es el componente de Spark para el procesamiento de grafos y la computación paralela de grafos.

Permite construir y analizar grafos dirigidos con propiedades en los vértices y aristas.

- Algoritmos de PageRank
- Componentes conectados
- Caminos más cortos
- Detección de comunidades



# Monitoreo y Depuración



## Interfaz Web

Spark proporciona una interfaz web en `http://localhost:4040` para monitorear aplicaciones en ejecución.



## Logs

Los logs de Spark contienen información detallada sobre la ejecución de jobs y errores.



## Métricas

Spark expone métricas a través de JMX que pueden ser recopiladas por sistemas de monitoreo.

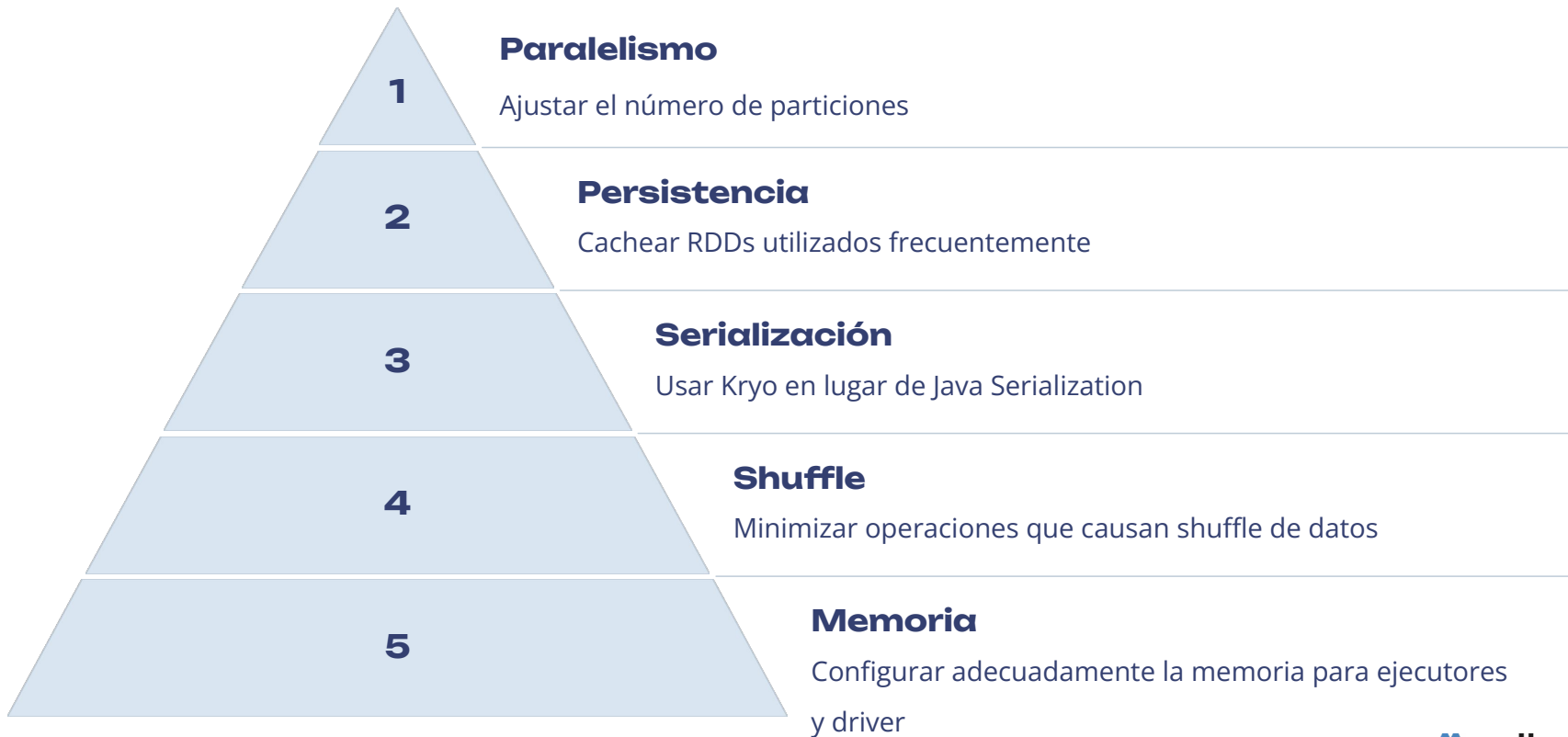


## Depuración

Utilizar `take()` o `collect()` con pequeñas muestras para verificar resultados intermedios.



# Optimización de Rendimiento



# Caso Práctico: Análisis de Logs

```
▶ # Cargar logs desde archivo
logs = sc.textFile("logs.txt")

# Filtrar líneas que contienen errores
errors = logs.filter(lambda line: "ERROR" in line)

# Extraer códigos de error
error_codes = errors.map(lambda line: line.split(" ")[3])

# Contar ocurrencias por código
error_counts = error_codes.map(lambda code: (code, 1)) \
    .reduceByKey(lambda a, b: a + b)

# Ordenar por frecuencia descendente
sorted_errors = error_counts.sortBy(lambda x: -x[1])

# Mostrar los 10 errores más comunes
print(sorted_errors.take(10))
```

Este ejemplo muestra cómo analizar logs para encontrar los errores más frecuentes.

# Caso Práctico: Análisis de Texto

## Caso Práctico: Análisis de Texto

```
▶ # Cargar texto desde archivo
text = sc.textFile("libro.txt")

# Dividir cada línea en palabras y convertirlas a minúsculas
words = text.flatMap(lambda line: line.lower().split(" "))

# Filtrar palabras vacías y dejar solo palabras alfabéticas
words = words.filter(lambda word: len(word) > 0 and word.isalpha())

# Contar frecuencia de cada palabra
word_counts = words.map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)

# Ordenar por frecuencia descendente
sorted_counts = word_counts.sortBy(lambda x: -x[1])

# Mostrar las 20 palabras más comunes
print(sorted_counts.take(20))
```

# Caso Práctico: Análisis de Datos CSV

Este ejemplo muestra cómo analizar datos de ventas utilizando Spark SQL.

[ 1 ]

```
# Importar SparkSession
from pyspark.sql import SparkSession

# Crear o recuperar la sesión de Spark
spark = SparkSession.builder.getOrCreate()

# Leer archivo CSV
df = spark.read.csv(
    "ventas.csv",
    header=True,
    inferSchema=True
)

# Registrar el DataFrame como tabla temporal
df.createOrReplaceTempView("ventas")

# Ejecutar consulta SQL para analizar ventas por región
result = spark.sql("""
    SELECT
        region,
        SUM(monto) AS total_ventas
    FROM ventas
    GROUP BY region
    ORDER BY total_ventas DESC
""")

# Mostrar resultados
result.show()
```

# Integración con Hadoop

## HDFS

Spark puede leer y escribir datos directamente desde HDFS.

```
rdd =  
sc.textFile("hdfs://namenode:  
8020/data/file.txt")
```

## YARN

Spark puede ejecutarse en clústeres YARN como gestor de recursos.

```
spark-submit --master yarn  
--deploy-mode cluster app.py
```

## Hive

Spark SQL puede leer y escribir tablas de Hive.

```
from pyspark.sql import  
HiveContext  
hive_context =  
HiveContext(sc)  
result =  
hive_context.sql("SELECT *  
FROM tabla")
```

# Despliegue de Aplicaciones Spark

## Desarrollo local

Usar modo local para desarrollo y pruebas iniciales.

```
spark-submit --master local[*] app.py
```

1

2

## Pruebas en clúster

Probar en un clúster pequeño antes de producción.

```
spark-submit --master spark://master:7077  
app.py
```

3

## Producción

Desplegar en clúster de producción con configuración optimizada.

```
spark-submit --master yarn --deploy-mode  
cluster --executor-memory 4g --num-executors  
10 app.py
```

4

## Monitoreo

Configurar alertas y monitoreo continuo.

# Configuración Avanzada

## Memoria

Ajustar `spark.executor.memory` y `spark.driver.memory` según las necesidades de la aplicación.

## CPU

Configurar `spark.executor.cores` para controlar el número de tareas por ejecutor.

## Shuffle

Optimizar `spark.shuffle.file.buffer` y `spark.reducer.maxSizeInFlight` para mejorar el rendimiento del shuffle.

## Serialización

Habilitar Kryo con `spark.serializer=org.apache.spark.serializer.KryoSerializer` para mejorar la eficiencia.

# Seguridad en Spark

## Autenticación

Configurar autenticación mediante Kerberos o SSL.



## Cifrado

Habilitar cifrado en tránsito con SSL/TLS.



## Autorización

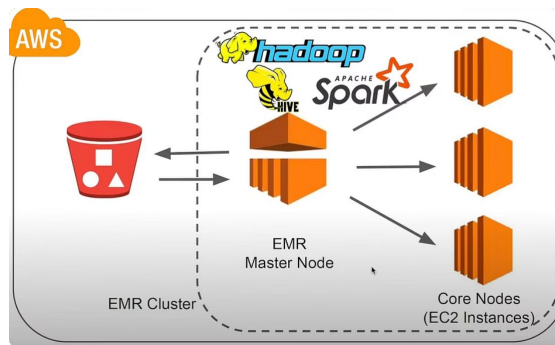
Implementar control de acceso basado en roles.

## Auditoría

Configurar registro de eventos para auditoría de seguridad.

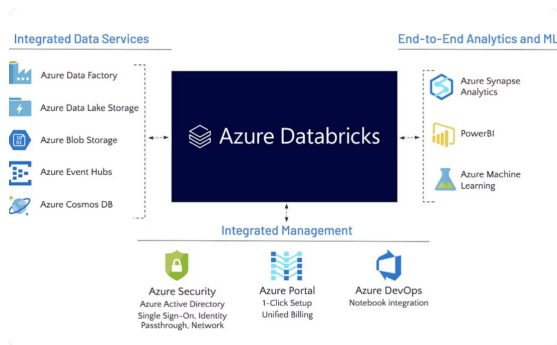


# Spark en la Nube



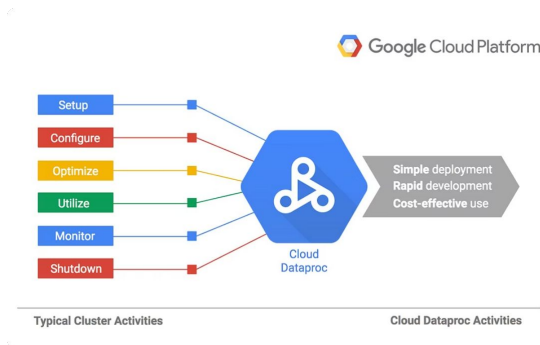
## AWS EMR

Amazon Elastic MapReduce ofrece Spark como servicio gestionado con integración con S3, DynamoDB y otros servicios de AWS.



## Azure Databricks

Plataforma optimizada para Spark en Azure con características avanzadas de colaboración y seguridad.



## Google Dataproc

Servicio gestionado de Spark en Google Cloud con integración con BigQuery, Cloud Storage y otros servicios de GCP.

# Comparación con Otras Tecnologías

## **Spark vs Hadoop MapReduce**

Spark es hasta 100 veces más rápido en memoria y tiene una API más rica y fácil de usar.

## **Spark vs Flink**

Flink está más orientado al procesamiento de streams verdaderos, mientras que Spark utiliza micro-batches.

## **Spark vs Dask**

Dask está más integrado con el ecosistema Python (NumPy, Pandas), mientras que Spark tiene un ecosistema más amplio para Big Data.

# Tendencias y Futuro de Spark



## Computación en la nube

Mayor integración con servicios en la nube y modelos serverless.



## IA y ML

Mejoras en las capacidades de machine learning y deep learning.



## Streaming

Procesamiento de streams más rápido y con menor latencia.



## Kubernetes

Mejor integración con Kubernetes para orquestación de contenedores.

# Recursos para Aprender Más



## Libros

- Learning Spark (O'Reilly)
- Spark: The Definitive Guide
- Advanced Analytics with Spark



## Cursos en línea

- Databricks Academy
- Coursera: Big Data Analysis with Spark
- edX: Distributed Computing with Spark



## Comunidad

- Stack Overflow
- Foros oficiales de Apache Spark
- Meetups locales de Spark

# Certificaciones de Spark

## **Databricks Certified Developer**

Certifica habilidades fundamentales en desarrollo con Apache Spark.

## **Databricks Certified Associate**

Nivel introductorio para profesionales que trabajan con Spark.

## **Cloudera CCA Spark and Hadoop Developer**

Certifica habilidades en desarrollo con Spark en el ecosistema Hadoop.

# Mejores Prácticas

## 1 **Particionar adecuadamente**

Ajustar el número de particiones según el tamaño de los datos y recursos disponibles.

## 2 **Persistir RDDs estratégicamente**

Cachear solo los RDDs que se reutilizarán múltiples veces.

## 3 **Minimizar shuffles**

Reducir operaciones que causan redistribución de datos entre nodos.

## 4 **Monitorear y ajustar**

Utilizar la interfaz web de Spark para identificar cuellos de botella y optimizar.

## 5 **Usar formatos de archivo eficientes**

Preferir formatos columnar como Parquet para mejor rendimiento analítico.

# Errores Comunes y Soluciones

## **OutOfMemoryError**

Aumentar memoria del driver/ejecutores o reducir paralelismo.

## **Serialización**

Asegurar que todas las clases utilizadas sean serializables.

## **Skew de datos**

Utilizar técnicas como salting para distribuir mejor los datos desbalanceados.

## **Spill a disco**

Optimizar configuración de memoria y shuffle para reducir escrituras a disco.

# Comunidad y Soporte

## Comunidad de código abierto

Apache Spark tiene una comunidad activa de desarrolladores y usuarios que contribuyen al proyecto y proporcionan soporte a través de:

- Listas de correo
- Foros de discusión
- Repositorio de GitHub
- Conferencias como Spark Summit

## Soporte comercial

Varias empresas ofrecen soporte comercial para Spark:

- Databricks (fundada por los creadores de Spark)
- Cloudera
- Hortonworks (ahora parte de Cloudera)
- MapR (ahora parte de HPE)





# Conclusión

El manejo de **RDDs en Spark** constituye la base del procesamiento de datos distribuido.

Comprender cómo crear, almacenar y transformar RDDs, junto con la diferencia entre **transformaciones** y **acciones**, permite construir soluciones escalables y eficientes.

El concepto de **lazy evaluation** y **linaje** ofrece tolerancia a fallos y optimizaciones automáticas, mientras que entender cómo se ejecuta un **Job Spark** brinda claridad sobre el flujo de trabajo interno del motor.

Dominar estos elementos es fundamental para desarrollar aplicaciones que manipulen grandes volúmenes de datos de forma eficiente, cumpliendo los objetivos planteados en entornos de **Big Data**.



# Live Coding

## ¿En qué consistirá la Demo?

Analizaremos de forma práctica cómo optimizar operaciones con RDD, aplicar variables broadcast y acumuladores, y leer/escribir datos en distintos formatos dentro de Spark.

1. Comparación práctica de rendimiento entre `groupByKey` y `reduceByKey`.
2. Uso de variables broadcast para compartir datos en el clúster.
3. Implementación de acumuladores para contadores globales.
4. Lectura de datos desde CSV, JSON y Parquet.
5. Escritura de resultados optimizados en formato columnar.



Ejercicio

# Optimización y extensión de RDDs

# Optimización y extensión de RDDs

## Contexto: 🙌

Vas a aplicar técnicas de optimización y funcionalidades avanzadas de Spark para mejorar el rendimiento del procesamiento distribuido.

## Consigna: 📝

1. Crea un RDD y compara el rendimiento de groupByKey y reduceByKey con un dataset mediano.
2. Implementa una variable broadcast para compartir un conjunto de datos de referencia.
3. Usa un acumulador para contar un evento específico durante el procesamiento.
4. Lee datos en formato CSV y escribe resultados en Parquet.

## Paso a paso: ⚙️

1. Preparar datos de prueba y configurar Spark.
2. Implementar las operaciones comparativas y medir tiempos.
3. Incorporar broadcast y acumuladores.
4. Guardar los resultados en el formato requerido.

**Tiempo** 🕒: 45 Minutos

¿Alguna consulta?





## ¡Ponte a prueba!

*Momento de ejercitación*

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.*

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

*Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.*

< ¡Muchas gracias! >

